SecureGate

A Distributed Multi-Factor Access Control System

CIS 5690 – Advanced Systems Project — Spring 2026

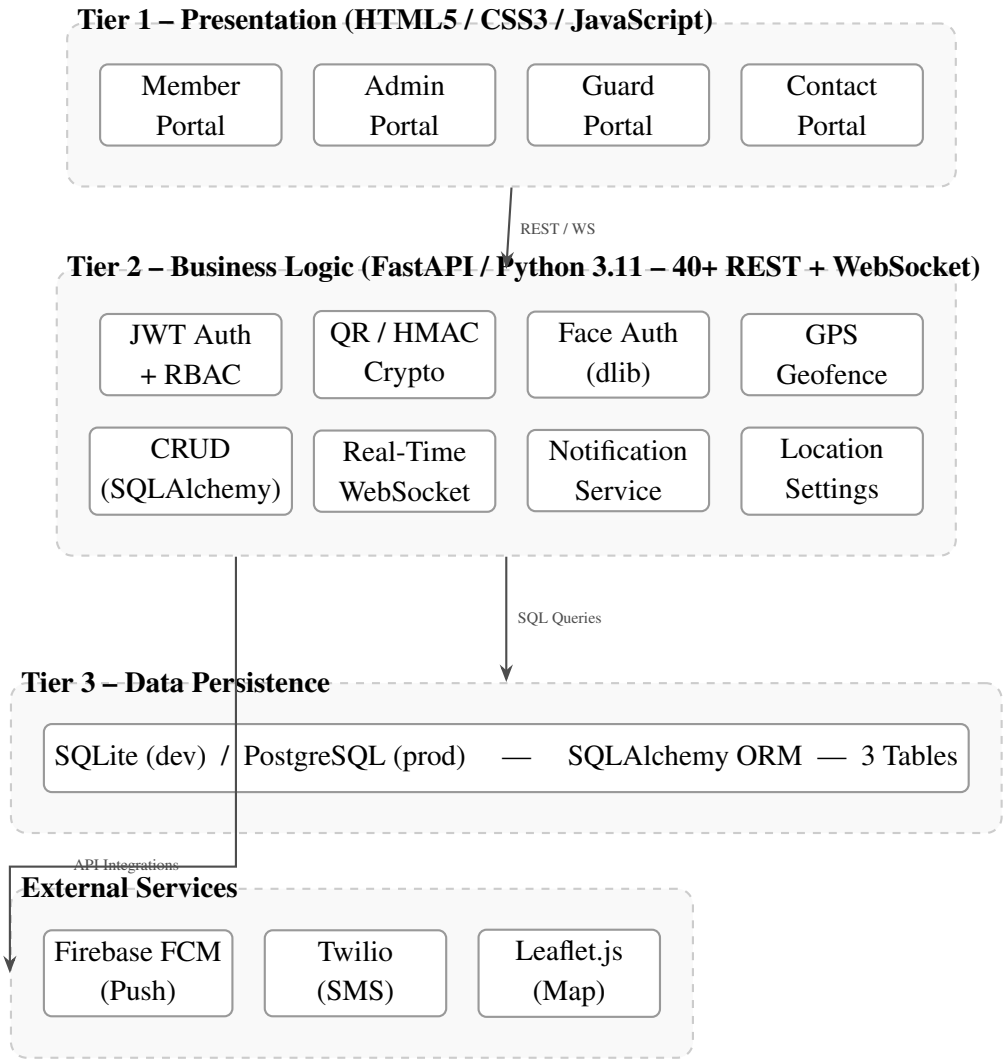
Project Overview

SecureGate is a full-stack, distributed, multi-factor digital gate management system for any secured facility: corporate offices, hospitals, residential communities, warehouses, or institutions. It replaces manual and card-based access with cryptographically signed QR passes, GPS geofencing, optional biometric verification, and real-time audit logging.

Problems Addressed:

- Physical ID cards are forgeable and shareable – solved by HMAC-SHA256 signed tokens
- No time-based restrictions – solved by 15-minute cryptographic expiry in every token
- Replay attacks – solved by one-time state machine: `pending -> approved -> used`
- No location enforcement – solved by server-side GPS geofencing (geopy + Shapely)
- No biometric identity proof – solved by dlib 128-D face encoding (optional)
- No real-time visibility – solved by WebSocket live dashboard and FCM/SMS notifications
- Paper logs, no auditability – solved by immutable `scan_logs` database table

System Architecture



QR Token Format:

`pass_id . user_id . unix_expiry . HMAC-SHA256_signature (32 hex chars)`

Database Schema (3 Tables):

- `users` – id, name, email, pwd_hash, role, member_id, department, face_encoding (JSON), face_registered, fcm_token, phone, contact_name, contact_phone, contact_fcm_token, valid_until
- `passes` – id, user_id (FK), reason, pass_type, status, qr_token, request_time, approved_by (FK), approved_time, expiry_time, used_time, used_by (FK), request_latitude, request_longitude, location_verified, location_distance_km
- `scan_logs` – id, pass_id (FK), user_id (FK), scanner_id (FK), scan_time, result, pass_type, details, emergency

User Roles

Role	Portal	Capabilities
Member	Member Portal	Request passes, generate instant GPS QR, register face, view history, emergency exit
Admin	Admin Portal	Approve/reject passes, live WebSocket dashboard, configure GPS boundary, analytics
Guard	Guard Portal	Scan QR via camera, face verify, view daily scan statistics
Contact	Contact Portal	View member access history without login, subscribe to SMS/push alerts

Technology Stack

Technology	Version	Role
Python	3.11+	Core backend language
FastAPI	0.115.2	REST API, auto Swagger docs, async support
Uvicorn	0.30.6	ASGI server – handles async and WebSocket
SQLAlchemy	2.0.36	ORM – same code runs on SQLite and PostgreSQL
Pydantic	2.9.2	Schema validation on all request and response models
PyJWT	2.9.0	JWT creation and verification (HS256)
Passlib + bcrypt	1.7.4	Password hashing, cost factor 12
HMAC-SHA256	stdlib	QR token signing and tamper verification
face-recognition	1.3.0	dlib 128-D face encoding and Euclidean comparison
OpenCV	4.8.1	Image pre-processing before face detection
Pillow	10.1.0	Image format, size, and dimension validation
Shapely	2.0.2	Polygon geofencing geometry
geopy	2.4.1	Geodesic GPS distance calculation
firebase-admin	6.3.0	FCM V1 API – multicast push notifications
Twilio	8.10.0	SMS delivery to contact persons
APScheduler	3.10.4	Background jobs: pass expiry, scheduled reports
html5-qrcode	CDN	In-browser camera QR scanning (Guard Portal)
QRCode.js	CDN	Client-side QR code rendering (Member Portal)
Leaflet.js	CDN	Interactive map for admin GPS boundary setup

Use Cases

UC01 – Member Authentication

- `POST /auth/login` – bcrypt.verify(password, hash) – JWT (HS256, 12h) issued
- Role in JWT drives portal redirect; `require_role()` enforces RBAC on every endpoint
- Alternate: wrong credentials – 401; wrong role for portal – redirect error

UC02 – Gate Pass Request with Admin Approval

- `POST /passes` – reason, pass_type, optional GPS stored; status = pending
- Firebase push sent to all admin FCM tokens on new request
- Admin approves: `make_qr_token(pass_id, user_id, ttl=15min)` – HMAC token generated
- DB: status = approved, qr_token, expiry_time stored; FCM + SMS sent to member
- Alternate: admin rejects – status = rejected, member notified via FCM + SMS

UC03 – Instant GPS-Verified Gate Pass

- `POST /passes/daily-entry` – browser Geolocation API sends GPS coordinates
- Server: `geodesic(facility_center, member_location).km <= radius + 0.050` (50m buffer)
- Inside boundary – pass auto-approved, HMAC QR generated instantly, no admin needed
- Alternate: outside boundary – HTTP 403; GPS permission denied – falls back to UC02

UC04 – Guard QR Verification at Gate

- `POST /verify` – token parsed into (pass_id, user_id, exp, sig)
- Four sequential checks: (1) HMAC re-computed and matched (tamper), (2) expiry check, (3) `status == "approved"` (state), (4) `used_time == None` (replay)
- On pass: `crud.mark_used()` – `crud.log_scan()` – WebSocket broadcast – FCM/SMS to contact
- Alternate: sig mismatch – invalid; expired – expired; status used – replay; pending – not approved

UC05 – Replay Attack Prevention

- Same QR scanned again – HMAC valid, expiry valid, but `used_time != None`
- System rejects: `ScanLog(result="replay")` inserted; guard sees rejection message
- Defense: HMAC (no forgery) + 15-min expiry + DB state (one-time) + immutable audit log

UC06 – GPS Geofence Validation

- Every pass request with coordinates calls `geofence.validate_location(lat, lon)`
- Circular: `geodesic(center, point).km <= radius + 0.050` via geopy
- Polygon: `Point(lon, lat).within(facility_polygon)` via Shapely
- `location_verified` and `location_distance_km` stored in every pass record

UC07 – Face Biometric Registration

- Pillow validates image (format, min 200×200px, max 5MB); dlib detects face
- `extract_face_encoding(image_bytes)` – 128-D float vector generated
- Vector serialized to JSON string, stored in `users.face_encoding`; no raw image retained
- Alternate: no face detected – error returned; multiple faces – largest face used

UC08 – Face Biometric Verification

- Guard submits live photo + member_id to `POST /api/verify_face`
- `compare_faces(stored_128D, live_128D, tolerance=0.6)` – Euclidean distance
- Confidence: High (0.0–0.4), Medium (0.4–0.6), No Match (>0.6); percentage returned

UC09 – Real-Time Notification Dispatch

- Six triggers: new request, approved, rejected, entry scan, exit scan, emergency exit
- Firebase: `MulticastMessage` – push to all registered FCM tokens simultaneously
- Twilio: `client.messages.create()` – SMS to E.164 numbers
- Graceful degradation: Firebase or Twilio failure does not affect core system operation

UC10 – Contact Person Alert Subscription

- No login required – contact enters member ID, last 30 days shown via `GET /api/parent/student_history`
- `POST /api/register_parent_fcm` – phone and FCM token saved to member record
- All future gate scans for that member trigger FCM push and SMS to the contact

UC11 – Admin Configures Geofence Boundary

- Leaflet.js map loads with current marker; admin drags to correct position, sets radius
- `POST /api/admin/location` – written to `location_settings.json`; effective immediately

UC12 – Real-Time Administrative Monitoring

- Admin connects to `ws://server/ws/logs`; server pushes 10 most recent scan records
- Every scan triggers `broadcast_new_scan()` to all connected admin sessions
- Supporting endpoints: `/api/logs/hourly`, `/api/logs/daily`, `/api/logs/search`

UC13 – Emergency Exit Request

- `POST /api/emergency_exit` – direct SQL INSERT; emergency = True, result = success
- No pass approval required; FCM + SMS to member and all admins; WebSocket broadcast triggered

CRUD Operations

Op	Table	Endpoint / Function	Triggered By
Create	passes	`POST /passes`	Member pass request
Create	passes	`POST /passes/daily-entry`	Member GPS instant pass
Create	scan_logs	`crud.log_scan()`	Every QR scan attempt
Create	scan_logs	`POST /api/emergency_exit`	Member emergency
Read	passes	`GET /passes`	Member / Admin
Read	users	`GET /auth/me`	All authenticated users
Read	scan_logs	`GET /scans`, `/scans/stats`	Guard dashboard
Read	scan_logs	`GET /api/logs/*`	Admin analytics
Read	scan_logs	`GET /api/parent/history/{id}`	Contact portal
Update	passes	`POST /passes/{id}/approve`	Admin approves
Update	passes	`POST /passes/{id}/reject`	Admin rejects
Update	passes	`crud.mark_used()`	System on scan success
Update	users	`POST /api/register_face`	Member face upload
Update	users	`POST /api/update_contact`	Member contact update
Update	users	`POST /api/register_fcm_token`	Any user device registration
Delete	—	Not exposed	Intentional – audit integrity

Current Project Status

Module	Status	Notes
JWT Authentication + RBAC	Complete	`require_role()` on all endpoints
HMAC-SHA256 QR Token (generate/verify)	Complete	`crypto.py`
Replay Attack Prevention	Complete	DB state + dual check in `/verify`
GPS Geofencing (circular + polygon)	Complete	`geofence.py`
Face Registration (dlib 128-D)	Complete	`face_auth.py`
Face Verification at Gate	Complete	Tolerance 0.6, 4 confidence levels
Admin Pass Management (approve/reject)	Complete	Status filter, stats cards
Real-Time WebSocket Dashboard	Complete	`realtime_logs.py`
Firebase FCM Push Notifications	Complete	6 event types, multicast
Twilio SMS Notifications	Complete	E.164 format, graceful degradation
Contact Person Portal (no login)	Complete	30-day history + subscription
Admin GPS Location Configuration	Complete	Leaflet.js + `location_settings.json`
Emergency Exit	Complete	Direct SQL INSERT, all admins notified
Guard Scan Statistics	Complete	Hourly counts, entry/exit split
Swagger Auto-Documentation	Complete	`/docs` – all 40+ endpoints
Pandas Analytics Layer	Pending	In progress
Bulk Data Generation (500+ members)	Pending	In progress
Unit Test Suite (pytest, 20+ cases)	Pending	In progress
UML Diagrams + Requirements Document	Pending	In progress

Completion Plan

Documentation:

- All 13 use case documents with preconditions, main flows, alternate flows, postconditions
- UML: Use Case diagram, Class diagram (10 classes), 3 Sequence diagrams, ER diagram
- Formal Requirements Document (functional + non-functional requirements)
- Project Notes: technology justification for all 19 stack components

Analytics and Big Data Layer:

- `analytics.py` (Pandas): peak access hours, 30-day trend, entry/exit ratio, GPS compliance rate
- `generate_bulk_data.py`: 500+ member records, 50,000+ scan log entries over 6 months
- New endpoints: `/api/analytics/peak-hours`, `/api/analytics/trend`, `/api/analytics/expo`
- Chart.js in Admin Portal: bar (peak hours), line (30-day trend), donut (entry/exit split)

Testing:

- `test_crypto.py` – 6 cases: token format, expiry, tamper detection, replay, user isolation

- `test_geofence.py` – 5 cases: inside, outside, invalid coordinates, buffer zone, polygon mode
- `test_face_auth.py` – 4 cases: encoding roundtrip, match, mismatch, confidence levels
- `test_auth.py` – 4 cases: valid login, wrong password, invalid token, role enforcement

Final Integration:

- End-to-end test all 13 use cases; PostgreSQL production migration verification
- Fix: CORS restriction for production, `asyncio.get_event_loop()` deprecation

Future Plans

Short-Term:

- React Native mobile app – native camera, offline QR display, native push notifications
- Multi-gate support – separate guard assignments and statistics per entry point
- Visitor pass management – one-time QR delivered via SMS to external guests
- Bulk member import via admin CSV upload

Long-Term:

- ML anomaly detection – Isolation Forest on scan logs to flag suspicious patterns
- RFID/NFC tap as hardware alternative to QR scanning
- Attendance analytics derived from entry/exit records per department
- Docker + Kubernetes for enterprise-scale horizontal scaling
- Multi-tenant SaaS – one instance serving multiple independent organizations